

Assignment 2: RAG and Iterative Tool Feedback for an RV32I Debugging Agent

ECEN 689: LLMs for Agentic Hardware Design & Security · Fall 2026

Student: Dhanush Adithya Lakkapuram Raguapthy · **UIN:** 837004069 · **Date:** 2 October 2026

Note: The directory structure of the submission, the location of the files for each task, and the steps to reproduce the results are explained in the `README.md` file in the work area (the submitted assignment folder).

1. Summary

I built a debugging agent that combines semantic retrieval (RAG), LLM-generated SystemVerilog, and Icarus Verilog feedback, and used it to assemble a single-cycle RV32I processor from four new modules (`pc_unit`, `branch_unit`, `memory_access`, `datapath`) and my four Assignment 1 modules (`alu`, `regfile`, `extend`, `controller`). All runs use gpt-5.4 through TAMU AI Chat.

Key results. Final processor: **114/114** unit vectors and **14/14** integration programs pass. All four seeded bugs were repaired in **one** repair each. Unseeded datapath ablation (3 runs/arm): pass rate **0/3** baseline, **1/3** RAG, **3/3** RAG+feedback. Seeded datapath: **0/3** RAG-only vs. **3/3** RAG+feedback (1, 2, 1 repairs).

The results show that project-specific context and the iterative tool-feedback loop each help, with feedback giving the larger gain. With three runs per arm they are descriptive only, not statistically significant, and they do not identify which retrieved chunk caused a given repair.

2. Configuration

Item	Setting / evidence
LLM / provider	gpt-5.4 via TAMU for every logged run (client default 4096 output tokens; no explicit temperature, seed, or dated snapshot recorded)
Embedding / index	sentence-transformers/all-MiniLM-L6-v2; FAISS inner product over normalized vectors (cosine similarity)
Chunking / retrieval	180 words per chunk, 30-word overlap; top-3 in Task 1 queries, top-5 in agent runs; retrieval happens once before the loop
Repair policy (fixed)	At most 5 repairs after candidate 0; stop on first pass; keep-best with rank <code>compile_error < sim_fail < pass</code>
HDL tool	Icarus Verilog 13.0 (version not stored in run JSONs)

Paths are relative to `A2_student/`. Task 4 metrics come from `ablation_experiment/`; the duplicate baseline JSONs in `logs/` are byte-identical copies, not extra trials.

3. Task 1: RAG knowledge base

Phase A (`rag/index.py`, `retrieve_chunks`) embeds the query with the same normalized encoder as the corpus, searches FAISS, and maps the returned row indices to `Chunk` objects in ranked order. It returns an empty list for non-positive k or an empty corpus, rejects an uninitialized index, caps k at the corpus size, and drops invalid indices. Scores are not preserved in logs (stored as `null`).

The index was built with `python -m rag.build_index --dataset rag_dataset --index-dir .rag_index`, giving **25 chunks from 23 documents** (9 module notes, 11 Verilog-bug notes, 2 RV32I references, 1 simulator-message note). The official `specs/` folder is supplied to the agent prompt directly and is *not* part of this index.

Five sample queries (top-3 results)

#	Query	Ranked chunks	Observation
1	controller pcsrc branch logic	1. <code>module_specs/controller_pcsrc.md</code> - Project controller's PCSrc logic from Branch[2:0], Zero, CmpResult, Jump (beq/bne/blt/bge cases). 2. <code>module_specs/module_interfaces.md</code> - Exact ports of pc_unit, branch_unit, memory_access and the A1 module signals. 3. <code>module_specs/datapath_integration.md</code> - pc_plus4 = pc + 4; pc_sel = (BranchD & take_branch) JumpD.	Strong topical match for branch redirection. However, the top chunk describes an A1-style multi-bit Branch/controller scheme that differs from this assignment's separate branch_unit, so its interface assumptions must not override the official spec.
2	datapath module interfaces ALUSrcBD immediate register	1. <code>module_specs/module_interfaces.md</code> - A1 module connections (controller outputs, ALU, extend, regfile ports). 2. <code>module_specs/datapath_integration.md</code> - ALU operand selects: alu_b = ALUSrcBD ? imm : rf_rd2; swapped-mux symptom. 3. <code>module_specs/module_interfaces.md</code> - A2 unit-module port lists (second, overlapping chunk).	Directly useful: exact ports and the ALUSrcBD ? imm : rf_rd2 rule. Two hits come from the same file but are different overlapping chunks.
3	pc_unit synchronous reset PC plus 4	1. <code>verilog_bugs/reset_polarity.md</code> - Reset polarity bug; example uses active-high asynchronous reset. 2. <code>module_specs/datapath_integration.md</code> - pc_plus4 = pc + 4 and pc_next mux. 3. <code>module_specs/module_interfaces.md</code> - pc_unit port list.	Mixed. PC+4 and port names are useful, but the top hit is an <i>asynchronous</i> reset example while the official pc_unit contract is synchronous.
4	branch signed unsigned comparison RV32I	1. <code>verilog_bugs/signed_comparison.md</code> - \$signed() cast for SLT; SLTU stays unsigned. 2. <code>riscv_spec/immediate_encoding.md</code> - B-type and other immediate bit layouts. 3. <code>riscv_spec/rv32i_alu_ops.md</code> - ALU operation table with signed vs. unsigned compare pitfalls.	Signed-cast and SLT/SLTU guidance is relevant to branch comparison; the immediate-encoding chunk is related but does not say how operands are compared.

5	memory access LB sign extension LBU byte enable	1. module_specs/memory_addressing.md - Word-addressed data RAM, addr[31:2] indexing. 2. riscv_spec/rv32i_alu_ops.md - ALU operation table (not load/store specific). 3. riscv_spec/immediate_encoding.md - Immediate layouts (not load/store formatting).	Weak coverage. The more relevant load_store_encoding.md chunk did not make the top three, so a larger k or query rewording would help.
---	--	---	--

Takeaway. Retrieval reliably returns topically related material (4 of 5 queries had a clearly useful top-3), but rank 1 can mislead (queries 1 and 3) and coverage can miss the best chunk (query 5). Retrieved text must therefore be treated as context, not as an overriding contract. Without relevance labels, I report no precision/recall figure. Full chunk texts are in `submission_artifacts/task1/query_results.txt`.

4. Task 2: Debugging agent

Phase	Implementation
B Initial prompt	Combines task and spec into a query. With RAG on, retrieves top-5 chunks, logs query/text/source/score, and appends them to the first user message.
C Classification	<code>classify_status</code> returns <code>pass</code> for exit code 0; for non-zero, compiler/elaboration/port/module/file diagnostics in <code>stdout+stderr</code> give <code>compile_error</code> , otherwise <code>sim_fail</code> .
D Candidate evaluation	Calls the LLM, writes the RTL, runs the provided checker, and records iteration, status, and return code.
E Keep-best	Keeps candidate 0, then replaces the best on equal or higher <code>_STATUS_RANK</code> ; a worse repair is discarded, the best RTL is restored on disk, and the record is marked <code>kept_best: true</code> .
F Repair message	Asks for a complete revised module and includes the current RTL plus <code>compact_feedback</code> (status, exit code, last 5000 characters of tool output).

Policy enforcement. The loop evaluates iteration 0 and at most iterations 1–5, and stops at the first pass. Baseline and RAG-only stop after candidate 0; only RAG+feedback enters repairs. For seeded runs, iteration 0 returns the seed verbatim with no API call. The same policy applies to every arm.

Smoke test. `pc_unit` with `rag_feedback` (`logs/pc_unit_smoke.json`): five chunks retrieved, iteration 0 `pass` (return code 0), zero repairs. It validates retrieval, generation and checking; the repair path is evidenced by Task 3.

Limitations. (i) The rank ordering compares broad status only, not the number of failing vectors. (ii) After a rollback the next prompt still references the rejected candidate; no logged run triggered a rollback, so this is untested. (iii) JSON logs store status and codes but not per-iteration RTL or full tool output, and `sim_fail` is the fallback for any unrecognized non-zero exit (e.g. code 255), so it does not prove a logic bug.

5. Task 3: Generation, integration and seeded repairs

Module	Log	Progression	Repairs
pc_unit	pc_unit_smoke.json	0: pass	0
branch_unit	branch_unit.json	0: pass	0
memory_access	memory_access.json	0: pass	0
datapath	datapath.json	0: sim_fail (255); 1: pass	1

All used rag_feedback, TAMU, gpt-5.4. The datapath was generated after the other seven modules were in rtl/. Integration pitfalls were port naming, using ALU Sum for addresses, immediate/register operand selection, PC+4 for jump links, and clearing target bit 0 for JALR. The first datapath failure's output was not saved, so its exact cause is unknown. All 14 sample programs pass (see Section 7); the passing set is saved in rtl_good_backup/.

Seeded bugs (RAG + feedback, each repaired in one repair)

Module	Injected fault	Replay failure	Log	Helpful retrieved chunk
pc_unit	PC + 32'd2 instead of +4	30/30	sim_fail → pass	datapath_integration.md (pc_plus4 = pc + 4); the spec also states this
branch_unit	BEQ/BNE conditions inverted	14/40	sim_fail → pass	controller_pcsrsrc.md (equal vs. not equal; interface differs from A2)
memory_access	LB zero-extends: {24'b0, byte}	6/44	sim_fail (255) → pass	load_store_encoding.md (LB signed vs. LBU unsigned)
datapath	ALU-B mux swapped: ALUSrcBD ? rd2 : ImmExtD	timeout on 01_add	sim_fail → pass	datapath_integration.md (exact mux direction and symptom)

The replay failure counts come from separate diagnostic replays, not the repair JSONs. All four bugs took one repair, so these runs show no difference in difficulty. Relevance of chunks was judged by reading them; the LLM's actual use of them was not measured. The seed files contain comments marking the defect, which reach the repair prompt, so success here is not diagnosis without hints.

6. Task 4: Ablation study

Design. Part A: unseeded datapath generation with baseline, RAG, and RAG+feedback, three runs each. Part B: seeded (bugs/datapath_buggy.sv) datapath with RAG and RAG+feedback, three runs each. Driver, spec, model, checker, and repair policy are identical across arms; rtl/datapath.sv was removed before each unseeded run. **Pass rate** = passes / runs. **Repairs** = index of the first passing candidate (0 for a pass at iteration 0); mean and range use successful runs only, and failed runs are excluded.

Part / arm	Runs	Passes	Pass rate	Failures	Successful n	Mean repairs	Range	Repairs per run
A: Baseline	3	0	0%	3	0	N/A	N/A	-
A: RAG	3	1	33.3%	2	1	0.00	0-0	0
A: RAG + feedback	3	3	100%	0	3	2.33	1-5	1, 5, 1
B: RAG	3	0	0%	3	0	N/A	N/A	-
B: RAG + feedback	3	3	100%	0	3	1.33	1-2	1, 2, 1

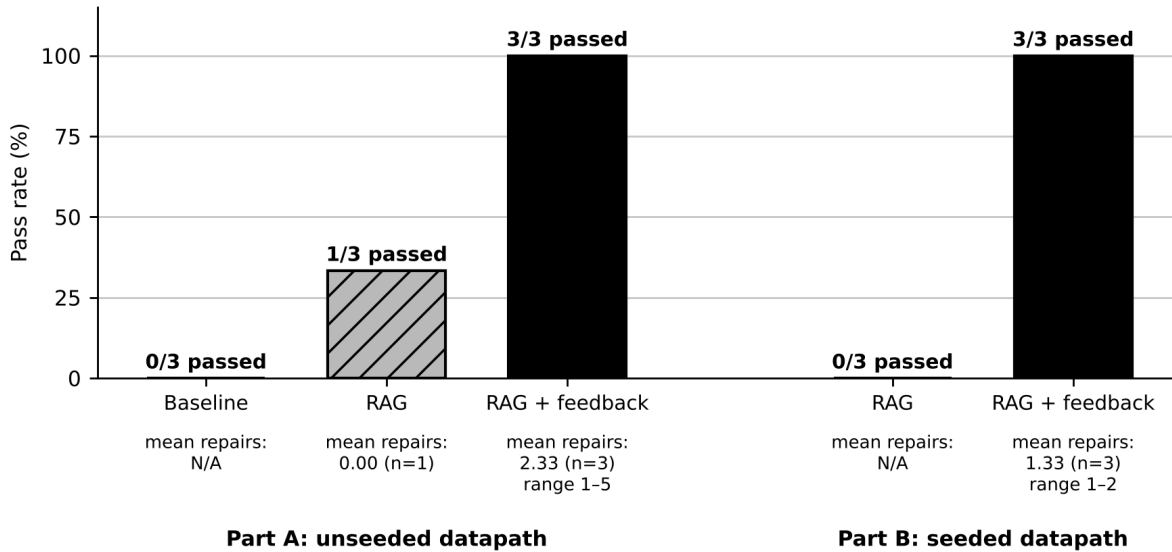


Figure 1. Pass rate per arm (3 runs each), with the mean number of repairs over successful runs below each bar (N/A = no successful run, not zero). Feedback raises the unseeded pass rate from 33% to 100%; one unseeded feedback run used the full 5-repair budget. Seeded RAG-only simply re-tests the unchanged seed.

Interpretation

- **Retrieval.** Baseline failed all three runs at compile time. Inspection of baseline run 1 shows guessed controller ports (`.op`, `.funct3`, `.zero`) and wrong ALU/memory port names, consistent with missing interface knowledge (the original compiler transcript was not saved). RAG raised the pass rate from 0% to 33%, a 33-point gain; its two failures were `sim_fail`, i.e. the code compiled.
- **Feedback.** RAG+feedback passed 3/3 (a further 67-point gain). It tested 10 candidates across three runs, against 3 in each no-feedback arm (16 candidates for Part A overall), so the gain reflects the whole feedback-and-retry procedure, not the feedback text alone.
- **Seeded control.** RAG-only makes no LLM call on a seeded run (the seed is returned and tested), so its 0/3 shows only that the seed fails, not that retrieval hurts. All three RAG-only snapshots matched the seed byte for byte. Feedback fixed the bug in 3/3 runs.
- **Variation.** Repair counts were 1, 5, 1 (Part A) and 1, 2, 1 (Part B). The mean of 2.33 hides one run that consumed the entire budget, so feedback improves outcomes without making generation deterministic. Part B run 2 had an intermediate `sim_fail` (code 255) whose cause is not recoverable from the logs.

- **Caveats.** Three runs per arm is a small sample; return code 255 failures lack saved output, and an earlier transient simulator file-access problem cannot be separated from functional failures; no run exercised rollback.

7. Task 5: Final processor and analysis

I restored the Task 3 processor with `cp rtl_good_backup/*.sv rtl/` (backup kept at the package root, not inside `rtl/`); a directory comparison shows the eight restored files are identical to the backup. `bash scripts/run_all_sample.sh` passes: **pc_unit 30/30, branch_unit 40/40, memory_access 44/44 (114 vectors)** and all **14** integration programs. A second run on a local copy (to avoid an intermittent SFTP-mount file-open issue) gave the same result with no failures.

Program	Signature	Cycles	Program	Signature	Cycles
01_add	0000001E	3	08_lbu_unsigned	000000FF	3
02_logic	000000AA	4	09_branch_taken	00000002	4
03_shift	00000001	3	10_branch_not_taken	00000003	4
04_signed_slt	00000001	3	11_jal	00000005	2
05_lui	12345678	2	12_jalr	00000009	3
06_word_load_store	00000056	4	13_auiop	00001001	2
07_lb_signed	FFFFFFFF	3	14_halfword	FFFF8001	4

Observed and expected signatures match for all programs; cycle counts are simulator observations, not timing data.

Final analysis

The four seeded bugs were localized, single-rule defects (PC increment, branch polarity, signed byte load, ALU operand mux), and each was repaired in one feedback-driven repair. The PC and load bugs map to one stated rule (PC advances by 4; LB sign-extends), making them the most direct. The branch and datapath bugs depend more on encodings and wiring, though they also took one repair. The retrieved integration and load/store notes were relevant, but the logs cannot show which chunk caused a given fix, and the hinting comments in the seed files weaken any claim of unaided diagnosis. Retrieval is helpful context, not a correctness guarantee.

In the ablation, RAG's main effect was supplying exact module interfaces: it removed the compile failures that sank every baseline run (0/3 to 1/3). Iterative feedback changed the outcome most, turning every failed first candidate into a pass in both unseeded (3/3) and seeded (3/3) settings, at the cost of extra attempts and run-to-run variation (1 to 5 repairs). The unseeded gain should be read as the value of retrieval plus a retry-with-feedback loop, as the arms differ in attempt count. A natural next step is to log full tool output and per-iteration RTL, and to rank `sim_fail` results by failing-vector count so that keep-best distinguishes finer progress.